



Bachelor's Degree in Computer Science, Networks, and Telecommunications Engineering

Final Course Project

Initial Report

2025-2026 Academic Year

EduCal – Academic Planning Application

Coordinators:

Filipe Freitas

Sandra Ferreira

Student:

Daniel Santos nº 51701

March 2026

1. Introduction

As part of the Bachelor's Degree in Computer Science, Networks, and Telecommunications Engineering, this report details the development of the final course project entitled EduCal – Academic Planning Application. The project, included in the 2025-2026 academic year, arises from the need to optimize the management of services involving critical planning, execution, and improvement activities.

Currently, these activities take place on dates set annually. However, there is no tool to facilitate the transition from one calendar year to the next, respecting specific criteria and deadlines. The current scenario for planning annual tasks is based on dense and complex spreadsheets (Excel). Although detailed, these tools are prone to filling errors and require an exhaustive manual effort to replicate the network of objectives, requesters, and instructions in the new school year.

The EduCal project aims to replace this manual process with an intelligent web-based application system. The central objective is to automate the migration of data from the current calendar to the following year, ensuring that fundamental business rules, such as the exclusion of weekends and holidays, are applied automatically. In this way, the aim is to centralize the workflow, eliminating redundancy and the risk of incorrect scheduling.

Given the nature of the project, its initial implementation is expected to take place within ISEL's services. To this end, the solution will be developed using modern technologies such as Next.JS and PostgreSQL and will be designed to be hosted on the institution's internal network.

2. Problem Description and Objectives

2.1. Problem Description

The current management of the academic calendar relies on dense and complex spreadsheets. Since there is no tool to automate the transition of dates between academic years, the process is manual, exhaustive, and prone to errors. In addition to dates, it is necessary to manually replicate a complex network of objectives and instructions, which increases the risk of incorrect scheduling.

2.2. Objectives

EduCal aims to replace manual effort with a web application. The main objectives are:

- **Baseline Creation:** Create an academic calendar from scratch as a base for the rest of the project and its functionalities.
- **Automation:** Automatically migrate the calendar from the current year to the next.
- **Application of Rules:** Ensure compliance with business rules, such as excluding holidays and weekends or events outside of school vacation periods.
- **Centralization:** Unify the workflow in a system that eliminates redundancies and ensures data integrity.

3. System Requirements

To ensure the EduCal application effectively addresses the identified problems and achieves its core objectives, a detailed set of requirements has been defined. These specifications serve as the technical blueprint for the development process, categorizing the system's needs into functional capabilities and non-functional quality standards.

3.1. Functional Requirements

Functional requirements outline the specific features, tasks, and behaviors the platform must provide to satisfy user needs. These define exactly what the system does, ranging from the fundamental management of academic events to the automated execution of the migration engine.

- **RF01 – Event and Occurrence Management:** The system must allow the creation and management of events that have multiple annual occurrences, each with its own descriptions and dates.
- **RF02 – Migration Engine:** It must be possible to transfer the event history from one academic year to the next, automatically validating the dates.
- **RF03 – Rule Configuration:** The system must allow flexible rules (e.g., working days) to be defined, which the migration engine must respect.
- **RF04 – Notification Templates:** The platform must allow the configuration of email templates associated with events for sending proactive notifications.
- **RF05 – Schedule Export (Optional):** The system must generate schedules in PDF format for institutional distribution.
- **RF06 – Profile Access Control (Optional):** Two types of users have been defined to ensure data security:
 - **Viewer:** Has read-only access to consult planning and schedules.
 - **Editor:** Has full permissions to perform CRUD operations, configure rules, and run the migration engine.

3.2. Non-Functional Requirements

Non-functional requirements specify the quality attributes, constraints, and performance criteria of the system. Rather than focusing on specific behaviors, they define how the system performs, ensuring that the application is secure, maintainable, and provides good user experience.

- **RNF01 – Data Integrity:** The rules for “business days” and holidays or school terms must be inviolable during the migration process.
- **RNF02 – Usability:** The interface must be intuitive and capable of presenting the necessary density of information.
- **RNF03 – Architecture and Scalability:** The system must be modular and scalable, ensuring fast access to extensive calendars.

4. System Architecture

To ensure the project's viability and security, the architecture was designed based on robust technologies that are widely adopted in the industry. The choice of an infrastructure hosted on ISEL's network ensures that the academic calendar service is integrated into the institutional ecosystem.

- **Frontend and Backend (Next.JS):** Use of a single framework to manage both the interface and the business logic and API routes.
- **Database (PostgreSQL):** Choice of a relational system to ensure the normalization and security of calendar data.
- **Hosting:** It is recommended to use a virtual machine within the ISEL network to host both the HTTP service and the database.

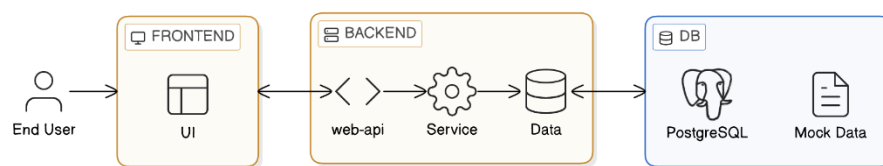


Figure 1 – System Architecture Diagram

As illustrated in Figure 1, the system follows a modern three-tier architecture to separate concerns and ensure maintainability:

- **Presentation Layer (Frontend):** The End User interacts with a responsive UI built in Next.js. This layer communicates bi-directionally with the backend via asynchronous requests, ensuring the interface remains fluid while processing data.
- **Logic Layer (Backend):** This layer is divided into three key components to organize the code effectively:
 - **Web-API:** Acts as the entry point, handling incoming requests and routing them to the correct logic.
 - **Service:** This is the "brain" of the application, where the Migration Engine and business rule validations (such as holiday and weekend checks) are executed.
 - **Data:** A dedicated layer for data access, abstracting the communication between the logic and the storage system.
- **Data Layer (DB):** A PostgreSQL relational database provides robust and persistent storage for events, occurrences, and system configurations. The architecture also accounts for Mock Data during the development and testing phases to ensure the system is reliable before connecting to live institutional data.

5. Schedule

As illustrated in Figure 2, the schedule is organized into three primary tracks—Deliverables, Backend, and Frontend, supplemented by two continuous activities that span the entire project lifecycle:

- **Continuous Activities:**
 - **Reporting:** Documentation and report writing are treated as ongoing tasks from the project's start on March 13th through to the final submission.
 - **Testing:** A dedicated testing phase runs parallel to all development activities to ensure system stability and requirement compliance throughout the build.
- **Deliverables and Milestones:**
 - **Initial Report Delivery:** Scheduled for the start of the project on March 13th.
 - **Intermediate Report Delivery:** Set for May 1st to assess progress.
 - **Final Report Delivery:** The project concludes with the final submission on July 11th.
- **Backend Development:**
 - **API Architecture and Setup:** This initial phase runs from mid-March to late April, establishing the system's core structure.
 - **Business Logic and Database:** Following the setup, the implementation of core engine logic and database integration occurs from early May to mid-June.
- **Frontend Development:**
 - **Base Interface:** UI design and basic prototyping take place during the first half of the project (mid-March to early April).
 - **Feature Development:** The primary build of application features occurs between early April and late June.
 - **Polishing:** A final refinement phase is scheduled for the last week of June to ensure a professional finish before the final delivery

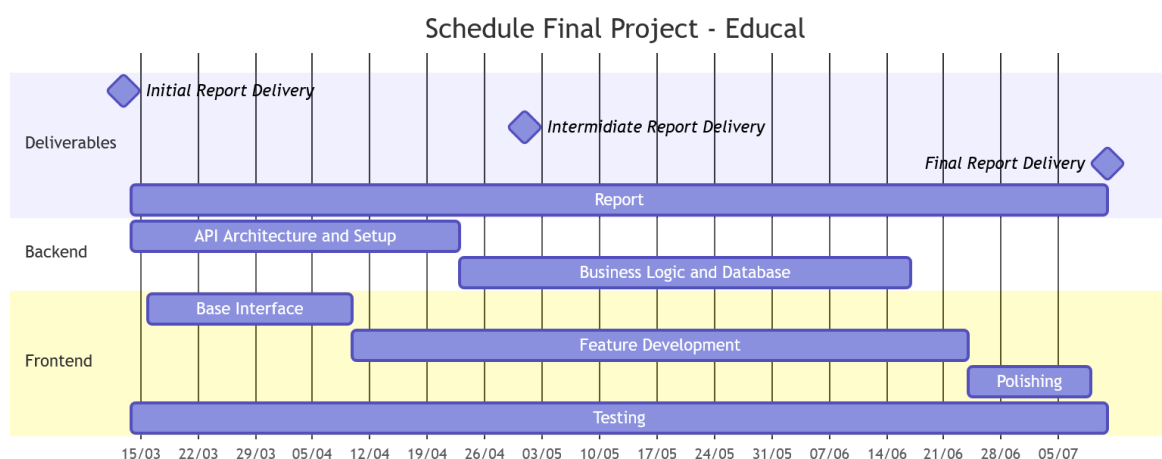


Figure 2 – Schedule

Report delivered on 13th March 2026